

Informazione

Attenzione!

Indicare quale tipologia di esame si intenda svolgere rispondendo alla domanda a scelta multipla seguente (domanda 1).

Si noti che è possibile leggere tutte o parte delle domande prima di effettuare la scelta e rispondere alla domanda 1.

Le domande dalla 2 alla 4 sono dedicate all'esame semplificato (traccia da 12pt). In particolare:

- la domanda 2 fa parte dell'esercizio da 2 punti.
- la domanda 3 fa parte dell'esercizio da 4 punti.
- la domanda 4 fa parte dell'esercizio da 6 punti.

Le domande dalla 5 in poi sono dedicate all'esame completo (traccia da 18pt). In particolare:

- la domanda 5 fa parte dell'esercizio da 4 punti.
- la domanda 6 fa parte dell'esercizio da 6 punti.
- la domanda 7 fa parte dell'esercizio da 8 punti.

Un apposito separatore fa da intervallo tra le domande del compito da 12pt e le domande del compito da 18pt.

Informazione

PER ENTRAMBE LE PROVE DI PROGRAMMAZIONE (18 o 12 punti)

- se non indicato diversamente, è consentito utilizzare chiamate a funzioni standard, quali ordinamento per vettori, funzioni su FIFO, LIFO, liste, BST, tabelle di hash, grafi e altre strutture dati, considerate come librerie esterne
- gli header file riferiti dal codice dovranno essere inclusi nella versione del programma allegata alla relazione
- i modelli delle funzioni ricorsive **non** sono considerati funzioni standard
- consegna delle relazioni, per entrambe le tipologie di prova di programmazione: entro il 30/01/2026, alle ore 23:59, mediante caricamento su Portale.
- le istruzioni per il caricamento sono pubblicate sul Portale nella sezione Materiale
- QUALORA IL CODICE CARICATO CON LA RELAZIONE NON COMPILI CORRETTAMENTE, VERRÀ APPLICATA UNA PENALIZZAZIONE. Si ricorda che la valutazione del compito viene fatta, senza la presenza del candidato, sulla base dell'elaborato svolto durante l'appello. Non verranno corretti i compiti di cui non sarà stata inviata la relazione nei tempi stabiliti.

Domanda 1
Completo

Indicare quale tipologia di esame si intende svolgere, selezionando UNA sola scelta tra a (12 punti) e b (18 punti).

Il tal caso occorrerà, per poter anticipare la correzione della prova scritta e del successivo eventuale orale, CARICARE LA RELAZIONE NEGLI ELABORATI entro il 29/1 alle 23:59.

Scegli una o più alternative:

- ☒ a. 12 Punti - Semplificato
- ☐ b. 18 Punti - Completo

Risposta parzialmente esatta.

Ignorare il feedback di questa domanda.

Le risposte corrette sono: 12 Punti - Semplificato, 18 Punti - Completo

Domanda 2

Completo

Punteggio
max.: 3,00

È dato un tipo struct node (e il relativo puntatore link) adatti a rappresentare nodi e puntatori in lista concatenata.

```
typedef struct node {  
    char *val;  
    struct node *next;  
} *link;
```

Il campo val punta a una stringa di proprietà del nodo. È data una lista (realizzata con questo tipo di dato) di stringhe, non ordinata e contenete eventuali dati ripetuti (anche più volte e non necessariamente contigui). Si consideri quale esempio, la lista seguente

->"Roma"->"Milano"->"Palermo"->"Roma"->"Torino"->"Genova"->"Roma"->"Palermo"

Si realizzi la funzione avente prototipo

link removeDupl(link h);

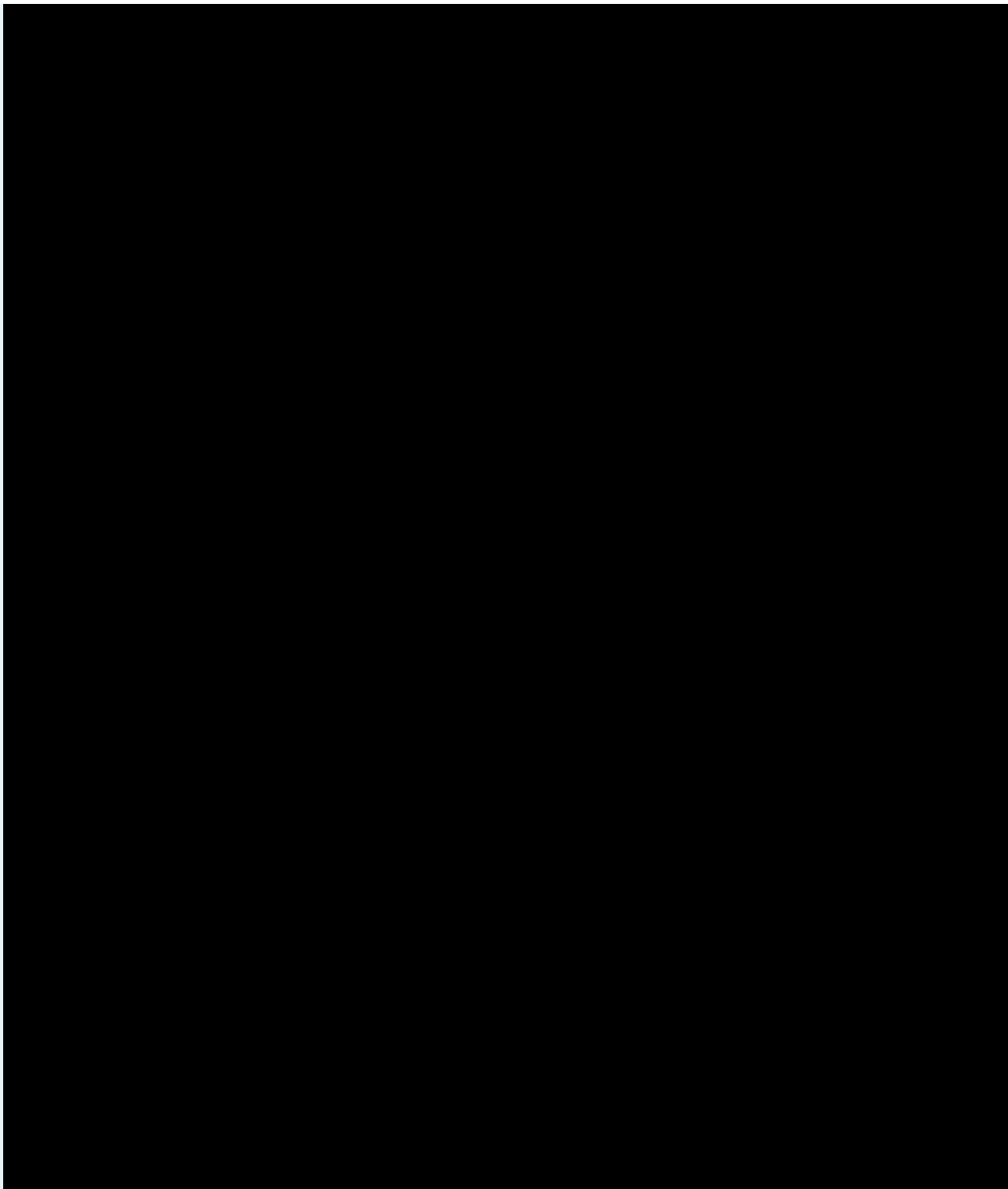
in grado di rimuovere dalla lista gli elementi duplicati. Di ogni stringa replicata più di una volta va mantenuta la prima istanza. I dati rimasti debbono conservare l'ordine originale. La lista precedentemente esemplificata, diventerebbe

->"Roma"->"Milano"->"Palermo"->"Torino"->"Genova"

Attenzione: non va generata una nuova lista ma occorre ritornare il puntatore alla lista modificata.

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

//scrivere qui il codice...

**Domanda 3**

Completo

Punteggio
max.: 4,00

È dato un tipo BST (ADT di prima classe). Si scriva una funzione che, dato un BST b , e due chiavi k_1 e k_2 , conti i nodi compresi tra le due chiavi (escluse dal conteggio) verificando anche se le due chiavi siano presenti. La funzione, avente prototipo

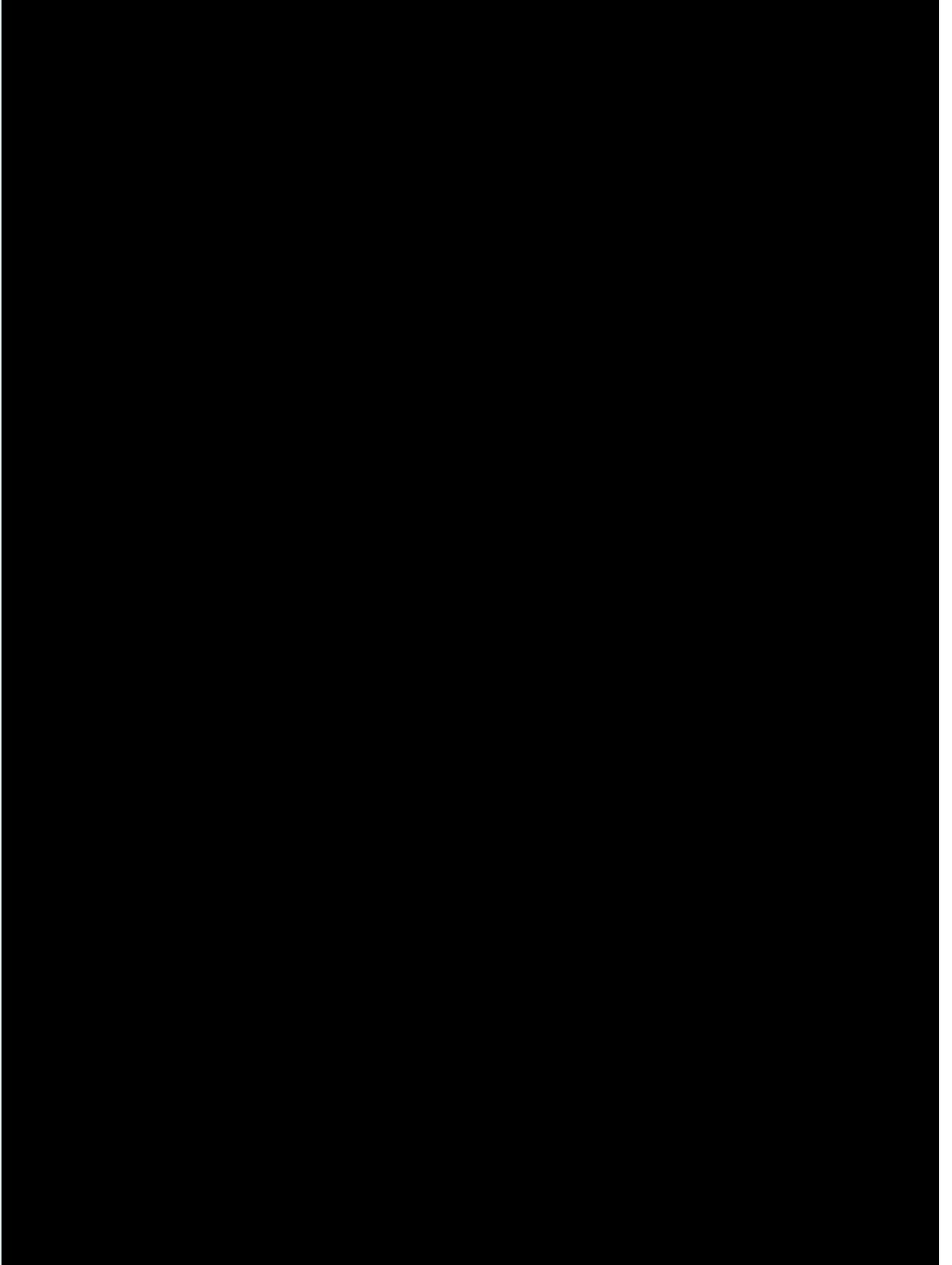
```
int BSTcountBetween(BST b, Key k1, Key k2);
```

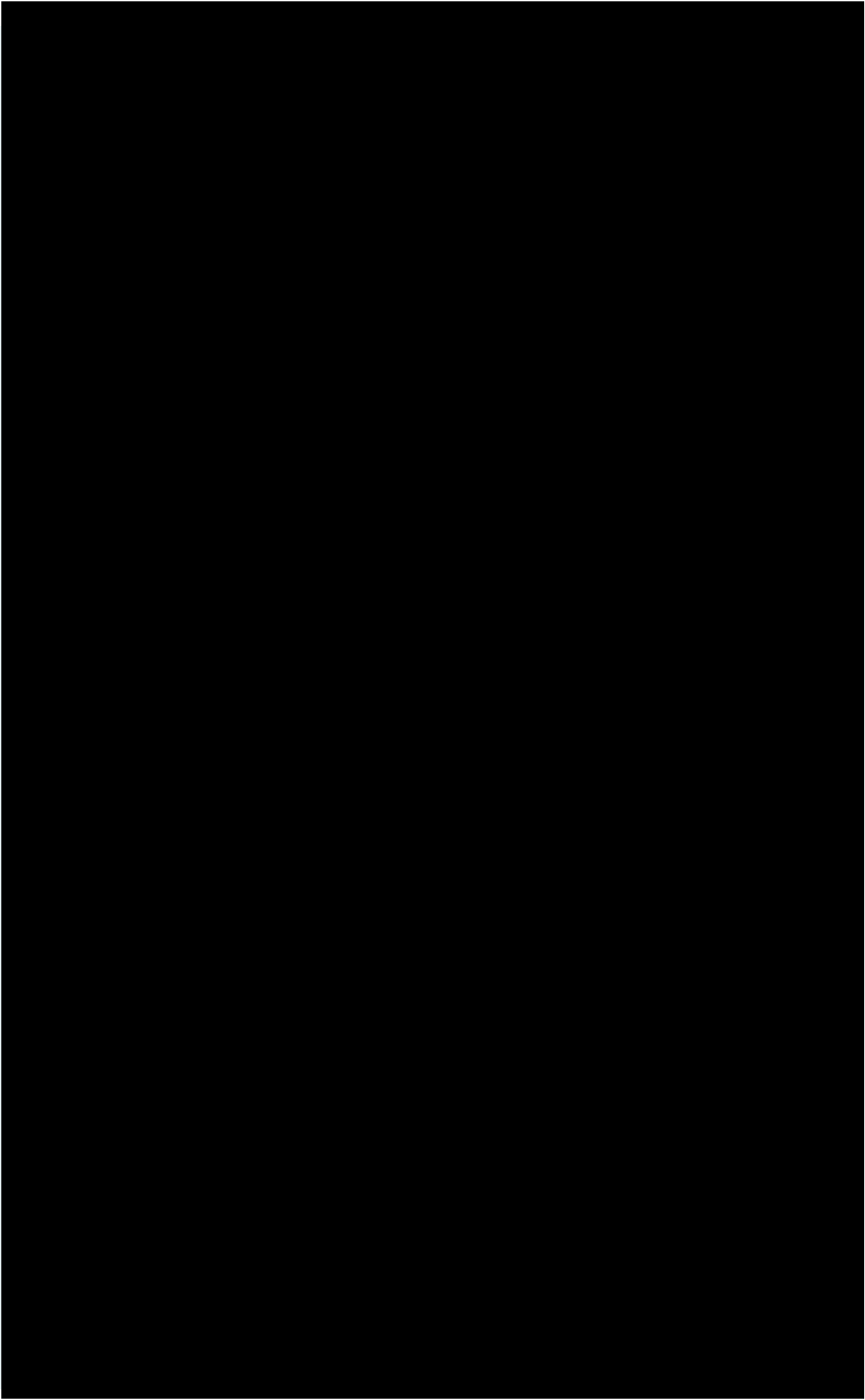
ritorni come risultato il conteggio, oppure -1 se una o entrambe le chiavi non sono presenti.

Attenzione: è preferibile (se ne terrà conto in fase di valutazione) una soluzione in grado di evitare la visita di parti dell'albero sicuramente da non conteggiare.

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

//scrivere qui il codice...





```
return -1; //se una o entrambe le chiave non sono presenti ritorno -1
}
return count;
}
```

Domanda 4

Completo

Punteggio
max.: 5,00

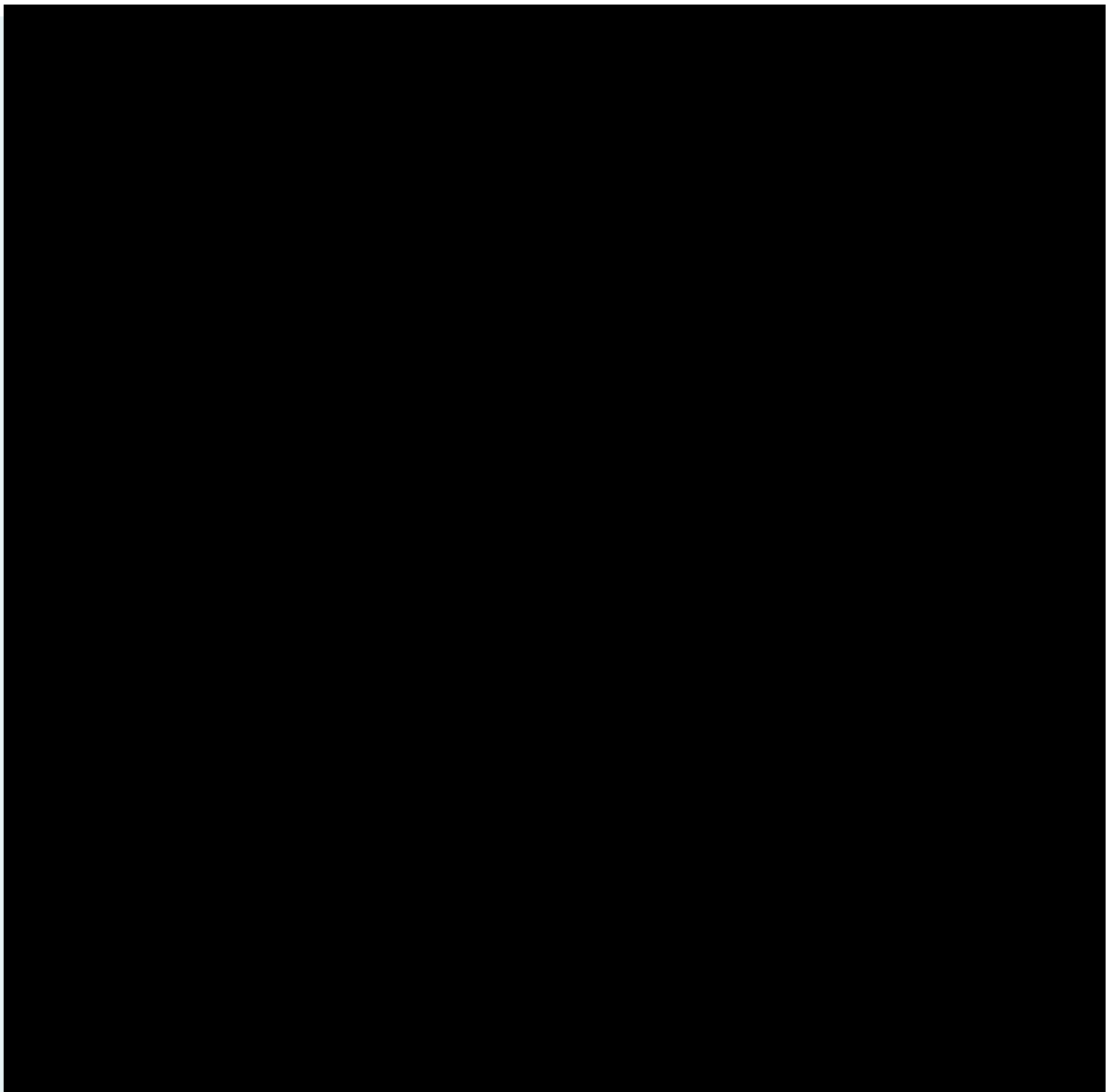
Date due stringhe s1 e s2, di lunghezza l1 e l2 (si supponga $l1 > l2$), verificare se rimuovendo l1-l2 caratteri da s1 si può ottenere una stringa uguale a s2. I caratteri possono essere rimossi in qualunque posizione delle stringa, non necessariamente contigui tra loro.

La funzione di verifica ritorni 0 per indicare falso e un numero !=0 per vero. Il prototipo è

```
int strContains(char *s1, char *s2);
```

Attenzione! Si consiglia l'uso degli spazi al posto delle tabulazioni per l'indentazione del codice, dal momento che il carattere TAB viene utilizzato per la navigazione della pagina da parte della piattaforma.

//scrivere qui il codice...



Informazione

PAGINA DI INTERMEZZO

La prova da 12 punti termina prima di questa pagina di intermezzo.

La prossima domanda rappresenta l'inizio della prova da 18 punti.

Informazione

Descrizione del problema

Un grafo diretto aciclico (DAG) descrive un albero genealogico, in cui i nodi rappresentano persone, caratterizzate da nome (si usa in questo contesto il termine nome per indicare la coppia cognome-nome), data di nascita e di morte (posta convenzionalmente a 0/0/0 se la persona è in vita), mentre gli archi diretti rappresentano relazioni genitore-figlio/a.

Si noti che sono possibili persone aventi lo stesso nome: in tal caso si può assumere che la data di nascita sia sufficiente a identificare una persona in modo univoco.

Si vogliono risolvere un problema di verifica e uno di ottimizzazione, così come specificato nel seguito.

Domanda 5

Risposta non data

Punteggio max.: 4,00

Struttura dati

Si supponga di utilizzare il tipo ADT visto a lezione per i grafi. Si indichi come va estesa la struttura dati per gestire nomi e date di nascita. È necessario fornire la definizione del tipo **Graph**. Va definito un tipo **Date** (quasi ADT), per rappresentare le date, e un tipo **Key**, che va usato per la tabella di simboli. Va infine scritta la funzione **KEYcmp**, che ritorni 0 per uguaglianza, oppure un risultato <0 o >0 a seconda del confronto tra le chiavi (si confrontano prima i nomi, poi le date di nascita). Non sono richieste le funzioni di gestione della tabella di simboli, né quelle di gestione del grafo, a meno che richiedano variazioni per la soluzione del problema proposto.

NON è richiesta la funzione di lettura da file.

Domanda 6

Risposta non data

Punteggio max.: 6,00

Problema di verifica

Date due persone, caratterizzate ognuna da nome e data di nascita, determinare se sono parenti di grado massimo k . Ci sono due possibilità:

- una delle due è antenata dell'altra e il cammino che le connette nel grafo ha lunghezza $\leq k$
- le persone hanno un/a antenato/a comune e le lunghezze dei due cammini che le connettono all'antenato/a, sommate tra loro non superano k .

La funzione abbia prototipo:

```
int relationshipK(Graph G, char *name1, Date d1, char *name2, Date d2, int k);
```


Domanda 7

Risposta non
data

Punteggio
max.: 8,00

Problema di ricerca e ottimizzazione

Trovare la persona con più discendenti fino al grado k . Il grado di un/a discendente è la sua distanza dall'antenato/a nel grafo. È sufficiente che la persona venga stampata. La funzione abbia prototipo
`void largestLineageK(Graph G, int k);`

ATTENZIONE: il numero di discendenti di grado k di una persona non è calcolabile a partire dai discendenti di grado k dei figli e delle figlie (e neppure, se lo si fosse calcolato, dei discendenti di grado $k-1$, perché potrebbero esserci discendenti comuni).